Assignment 5
Advanced Multi-Client Chat Server using TCP

Name: Aman
Roll No: 241001160007

1. Objective

The objective of this assignment is to enhance the multi-client TCP chat application developed in Assignment 4 by implementing advanced client management, private messaging, online user listing, persistent chat history, performance evaluation, graph generation, and packet-level verification using Wireshark. The project demonstrates practical concepts of TCP socket programming, multithreading, client-server communication, and network performance analysis.

2. System Architecture

The application follows a centralized client-server architecture. A single TCP server accepts connections from multiple clients over port 5000. Each connected client is handled by a separate thread. The server maintains information about every connected client, routes broadcast and private messages, stores chat history in CSV format, and records performance statistics.

Architecture:
• h1 – Chat Server
• h2 – Client A
• h3 – Client B
• h4 – Client C
• h5 – Client D

Mininet Topology: sudo mn --topo single,5

3. Design Improvements over Assignment 4

- Enhanced client management by storing username, IP address, port number, login time, and online/offline status.
- Join and leave notifications are broadcast to all connected clients.
- Private messaging implemented using '/msg <username><message>'.

- Online user list implemented using '/list'.
- Persistent chat history stored in chat_history.csv.
- Previous five messages displayed when a user reconnects.
- Performance metrics stored in performance_results.csv.
- Automatic graph generation for experimental analysis.

4. Program Design

The server uses Python socket programming and multithreading. A dictionary maintains active client information. Incoming messages are classified as broadcast, private messages, or commands. Broadcast messages are forwarded to all connected users except the sender, while private messages are routed only to the intended recipient. Chat history is written to a CSV file, and performance metrics such as delivery delay and throughput are calculated during execution.

- Main Components:
    - server.py
    - client.py
    - chat_history.csv
    - performance_results.csv
    - generate_graphs.py

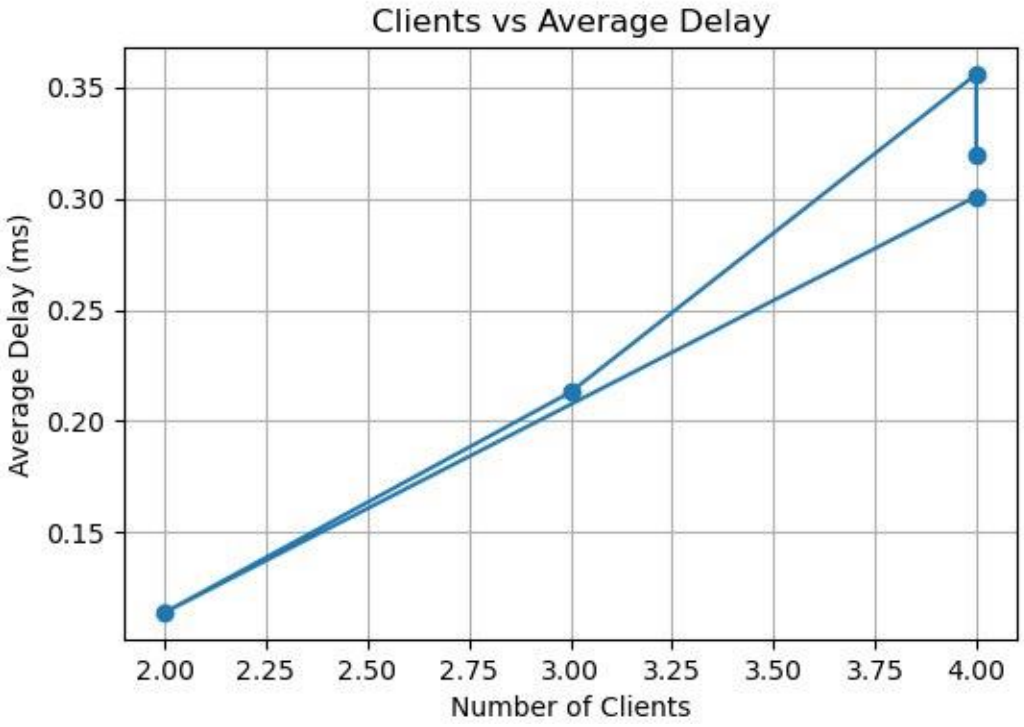
5. Experimental Setup

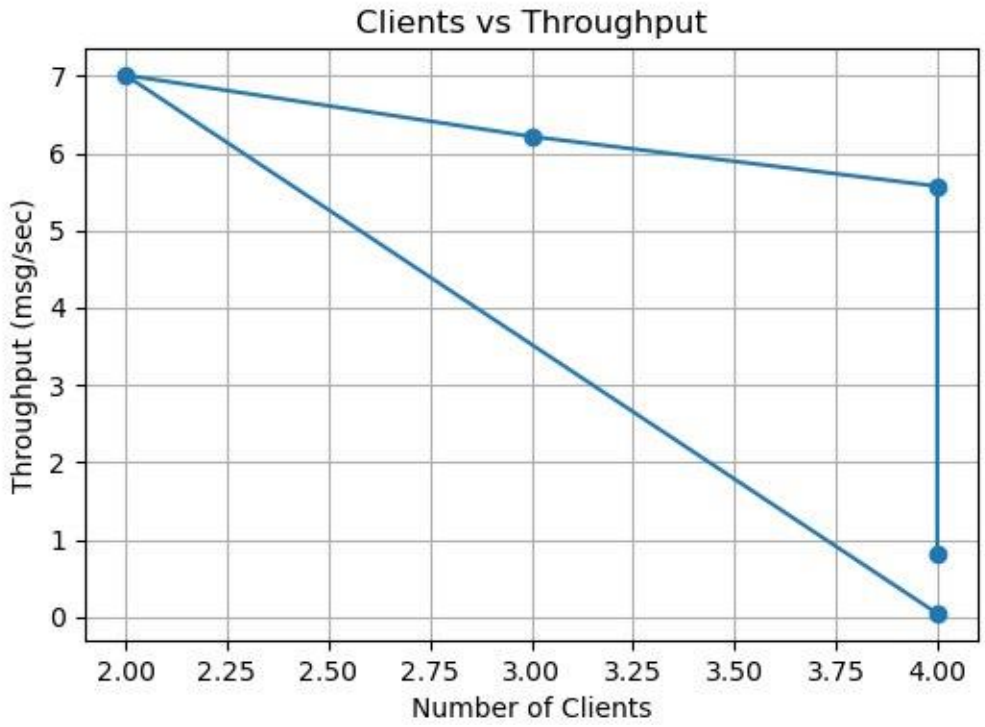
The experiment was performed using Mininet with one server and four clients. Connectivity was verified using nodes, net, and pingall commands. Wireshark captured TCP traffic using the display filter 'tcp.port == 5000'. Performance evaluation was conducted using 2, 3, and 4 clients, with each client transmitting 50 messages.

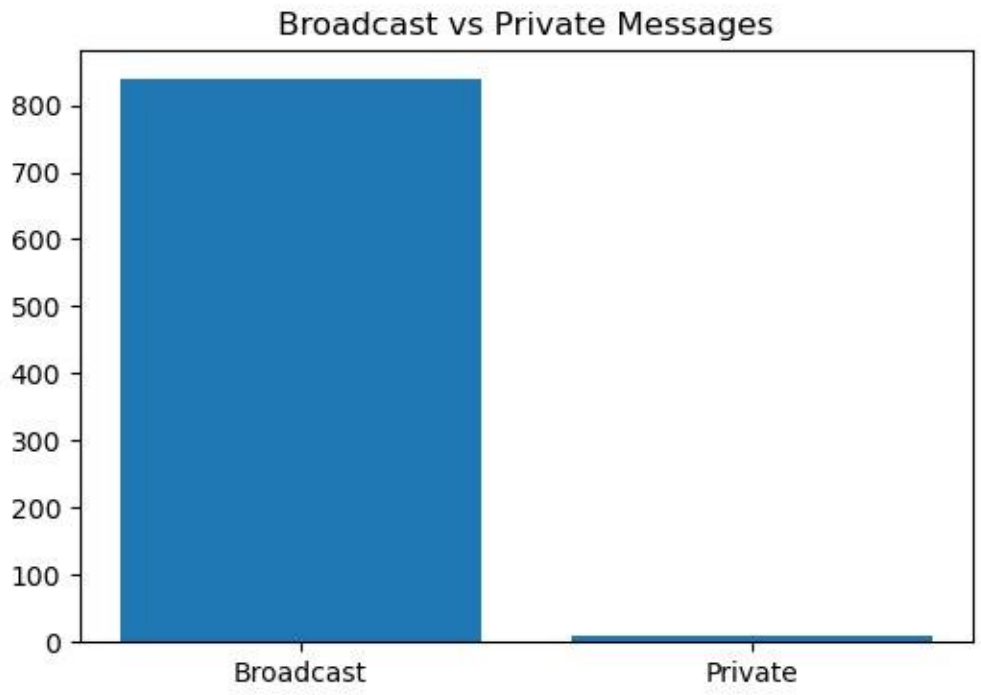
6. Results

Insert the generated performance_results.csv table here. Discuss the measured average delivery delay, throughput, number of broadcast messages, and number of private messages. Mention that throughput increased with the number of clients while average delay also increased because the server handled additional concurrent traffic.

7. Graph Analysis







Analysis:
The average delivery delay increased gradually as the number of clients increased because the server processed more concurrent messages. Throughput also increased because more messages were transmitted per unit time. Broadcast traffic was significantly higher than private traffic because most communication during testing used broadcast messages.

8. Wireshark Verification

Wireshark was configured with the filter 'tcp.port == 5000'. Packet captures verified TCP connection establishment (SYN, SYN-ACK, ACK), broadcast communication (PSH, ACK), private messaging (PSH, ACK between server and intended recipient), client disconnection (FIN, ACK), and TCP connection termination using the standard four-way handshake. Insert screenshots for each captured event.

9. Reflection Questions

1. Why is server-side routing necessary for private messaging?

Server-side routing ensures that private messages are delivered only to the intended recipient. The server identifies the destination user, validates that the user is connected, and forwards the message securely without exposing it to other clients.

2. How does maintaining user state improve the application?

Maintaining user state enables the server to track active users, login information, connection status, and reconnect history. It also supports features such as online user listing and accurate message routing.

3. How would the application scale to 100 users?

To support approximately 100 users, asynchronous I/O or thread pools could replace one-thread-per-client, authentication could be added, chat history could be stored in a database, and load balancing or distributed servers could improve scalability.

4. Which Wireshark packets verified private messaging?

Private messaging was verified using TCP PSH, ACK packets carrying the application payload from the server to the intended recipient. The absence of identical packets to other clients confirmed correct routing.

5. What improvements would you implement next?

Future improvements include TLS encryption, user authentication, GUI support, group chat, file transfer, emojis, offline messaging, database-backed history, and improved scalability.

10. Conclusion

The assignment successfully extended the Assignment 4 application into a feature-rich multi-client TCP chat server. Enhanced client management, private messaging, online user listing, persistent chat history, performance evaluation, graph generation, and Wireshark verification were implemented successfully. The project strengthened practical knowledge of socket programming, concurrent systems, and TCP communication while demonstrating the evolution of a network application from a basic implementation to a more complete client-server solution.